Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №6

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Побудування програмної системи з множини об’єктів, керованих повідомленнями»

Виконав:

студент групи ІМ-о51
Мосендз Белла Яківна
Номер в списку - 9

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab6.

2. Написати вихідні тексти усіх програм-компонентів згідно варіанту завдання.

3. Скомпілювати вихідні тексти і отримати виконувані файли програм.

4. Перевірити роботу програм. Налагодити взаємодію програм.

5. Проаналізувати та прокоментувати результати та вихідні тексти програм.

6. Оформити звіт

Вимоги щодо організації системи

Для виконання лабораторної роботи потрібно створити три незалежні програми. На відміну від попередніх лабораторних, де декілька файлів утворювали одну програму (executable), це повинні бути три незалежні програми. Легкий спосіб перевірити, чи це дійсно окремі програми — відкрити Диспетчер завдань Windows (чи його відповідний аналог в іншій ОС).

Головний проект – програма Lab6 має бути менеджером, який керує двома іншими програмами – Object2 та Object3. Програма Lab6 повинна автоматично, без участі користувача налагоджувати співпрацю та виконувати обмін повідомленнями з програмами Object2 та Object3 для виконання потрібного завдання згідно наведеному вище алгоритму.

У кожній програмі передбачити автоматичну обробку потрібних повідомлень і переходів з одного стану в інший.

Далі вимоги щодо функціонування системи об'єктів.

1. Для початку роботи користувач програми вибирає потрібний пункт меню програми Lab6. Далі з’являється вікно діалогу, у якому потрібно ввести параметри згідно варіанту завдання. У вікні діалогу користувач натискує кнопку "Так" (або "Виконати") і на цьому місія користувача закінчується – далі він тільки спостерігає, як програма сама автоматично виконає усе, що потрібно для отримання результату. Виклик інших програм – Object2 та

Object3 головна програма Lab6 повинна робити без участі користувача.

2. Обмін повідомленнями та масивами даних між програмами Lab6, Object2 та Object3 повинен відбуватися автоматично, без участі користувача. Програма Lab6 повинна автоматично у певній послідовності знаходити та викликати програми Object2 та Object3.

3. У результаті одного сеансу роботи користувач повинен бачити головні вікна програм Object2 та Object3, у яких відображатимуться потрібні результати відповідно варіанту завдань. Для цього вікна програм повинні автоматично розташуватися так, щоб усі результати було видно. Програма Lab6 повинна залишатися у активному стані, щоб користувач мав можливість повторно виконати роботу.

4. Передбачити варіанти успішної роботи у випадках, коли програми Object2 та Object3 (одна або обидві) до того вже були викликані.

5. По завершенні роботи програми Lab6 повинні автоматично завершуватися і програми Object2 та Object3.

Вибір варіанту завдання

Номер варіанту завдання згідно формулі Номер варіанту = Ж mod 4 де: Ж – номер в списку студентів в журналі.

1	1. Користувач вводить значення параметрів **n**, **Min**, **Max** у діалоговому вікні. 2. Програма викликає програми **Object2**, **Object3** і забезпечує обмін повідомленнями для передавання та отримання інформації.	1. Створює матрицю **n×n** цілих **(int)** чисел у діапазоні **Min – Max** 2. Показує числові значення у власному головному вікні 3. Записує дані в Clipboard Windows у текстовому форматі	1. Зчитує дані з Clipboard Windows 2. Відображає значення детермінанту матриці у власному головному вікні

Main file: MainWindow.xaml.cs (dir Lab6)

```
using System.Windows;

using System.IO;

using System.IO.Pipes;

using System.Diagnostics;

using System.ComponentModel;



namespace Lab6;



public partial class MainWindow : Window

{

    private bool _isShuttingDown;

    public MainWindow()

    {

        InitializeComponent();

    }



    private async void Run_Click(object sender, RoutedEventArgs e)

    {

        var dialog = new ParametersDialog

        {

            Owner = this

        };
```

```
        // поки користувач не ввів числа, головне вікно заблоковане
        if (dialog.ShowDialog() != true)
        {
            return;
        }

        int n = dialog.N;
        int min = dialog.MinValue;
        int max = dialog.MaxValue;

        try
        {
            statusText.Text = "Стан: запуск Object2...";

            await EnsureComponentAsync("Object2");

            statusText.Text = "Стан: запуск Object3...";

            await EnsureComponentAsync("Object3");

            statusText.Text = "Стан: розташування вікон...";

            await ArrangeWindowsAsync();
```

```
            statusText.Text = "Стан: генерація матриці...";

            string? response2 = await SendCommandAsync(
                "Lab6_Object2",
                $"GENERATE|{n}|{min}|{max}"
            );

            if (response2 != "READY")
            {
                statusText.Text = $"Помилка Object2: {response2 ?? "Немає відповіді"}";
                return;
            }

            statusText.Text = "Стан: обчислення визначника...";

            string? response3 = await SendCommandAsync(
                "Lab6_Object3",
                "CALCULATE"
            );

            if (response3 == "READY")
            {
                statusText.Text = "Готово! Визначник обчислено.";
```

```
            }
            else
            {
                statusText.Text =
                    $"Помилка Object3: {response3 ?? "Немає відповіді"}";
            }
        }
        catch (Exception ex) when (
            ex is IOException or
            TimeoutException or
            System.ComponentModel.Win32Exception)
        {
            statusText.Text = $"Помилка: {ex.Message}";
        }
    }

    private void Exit_Click(object sender, RoutedEventArgs e)
    {
        Close();
    }

    private static async Task<string?> SendCommandAsync(string pipeName,
string command)
    {
```

```
        using var pipe = new NamedPipeClientStream(
            ".",
            pipeName,
            PipeDirection.InOut,
            PipeOptions.Asynchronous
        );

        await pipe.ConnectAsync(2000);

        using var reader = new StreamReader(pipe);
        using var writer = new StreamWriter(pipe)
        {
            AutoFlush = true
        };

        await writer.WriteLineAsync(command);

        return await reader.ReadLineAsync();
    }

    private static string GetExecutablePath(string name)
    {
        // шлях до лаб 6
        var output = new DirectoryInfo(AppContext.BaseDirectory);
```

```
    string framework = output.Name;
    string configuration = output.Parent!.Name;

    string labFolder = Path.GetFullPath(
        Path.Combine(AppContext.BaseDirectory, "..", "..", "..", "..")
    );

    return Path.Combine(
        labFolder,
        name,
        "bin",
        configuration,
        framework,
        $"{name}.exe"
    );
}

private static async Task EnsureComponentAsync(string name)
{
    string pipeName = $"Lab6_{name}";

    // виконується тільки якщо процес ще не запущений,
    // їх можна буде використовувати не відкриваючи повторно
```

```
        if (Process.GetProcessesByName(name).Length == 0)
        {
            string path = GetExecutablePath(name);

            if (!File.Exists(path))
            {
                throw new FileNotFoundException(
                    $"Не знайдено виконуваний файл {name}.exe"
                );
            }

            Process.Start(new ProcessStartInfo
            {
                FileName = path,
                WorkingDirectory = Path.GetDirectoryName(path)!,
                UseShellExecute = true
            });
        }

        for (int attempt = 0; attempt < 15; attempt++)
        {
            try
            {
                string? response = await SendCommandAsync(
```

```
                    pipeName,
                    "PING"
                );

                if (response == "PONG")
                {
                    return;
                }
            }
            catch (Exception ex) when (
                ex is IOException or TimeoutException)
            {
                // Компонент ще запускається
            }

            await Task.Delay(300);
        }

        throw new IOException($"{name} не відповідає на повідомлення.");
    }

    private async Task ArrangeWindowsAsync()
    {
        // повертає область моніторy
```

```
        Rect area = SystemParameters.WorkArea;
        int margin = (int)(area.Height * 0.1);

        int x = (int)area.Left;
        int y = (int)area.Top + margin;
        int width = (int)(area.Width / 3);
        int height = (int)area.Height - 2 * margin;

        string? response2 = await SendCommandAsync(
            "Lab6_Object2",
            $"MOVE|{x + width}|{y}|{width}|{height}"
        );

        string? response3 = await SendCommandAsync(
            "Lab6_Object3",
            $"MOVE|{x + width * 2}|{y}|{(int)area.Width - width *
2}|{height}"
        );

        if (response2 != "OK" || response3 != "OK")
        {
            throw new IOException(
                $"Object2: {response2 ?? "Немає відповіді"}, " +
                $"Object3: {response3 ?? "Немає відповіді"}"
```

```
            );
        }

        WindowState = WindowState.Normal;

        Left = x;
        Top = y;
        Width = width;
        Height = height;
    }

    private static async Task StopComponentAsync(string name)
    {
        try
        {
            string? response = await SendCommandAsync(
                $"Lab6_{name}",
                "QUIT"
            ).WaitAsync(TimeSpan.FromSeconds(3));

            if (response != "BYE")
            {
                Debug.WriteLine($"{name}: {response}");
            }
```

```
        }
        catch (Exception ex) when (
            ex is IOException or TimeoutException)
        {
            Debug.WriteLine($"{name}: {ex.Message}");
        }
    }

    private async void MainWindow_Closing(object? sender, CancelEventArgs
e)
    {
        e.Cancel = true;

        // захист від подвійного натискання
        if (_isShuttingDown)
        {
            return;
        }

        _isShuttingDown = true;
        IsEnabled = false;

        try
        {
```

```
            await StopComponentAsync("Object2");
            await StopComponentAsync("Object3");
        }
        finally
        {
            Closing -= MainWindow_Closing;
            Close();
        }
    }
}
```

ParametersDialog.xaml.cs (dir Lab6)

```
using System.Windows;
using Lab6.Shared;

namespace Lab6;

public partial class ParametersDialog : Window
{
    public int N { get; private set; }
    public int MinValue { get; private set; }
    public int MaxValue { get; private set; }

    public ParametersDialog()
```

```
    {
        InitializeComponent();
    }

    private void Confirm_Click(object sender, RoutedEventArgs e)
    {
        if (!int.TryParse(nBox.Text, out int n) ||
            !int.TryParse(minBox.Text, out int min) ||
            !int.TryParse(maxBox.Text, out int max))
        {
            MessageBox.Show("Введіть цілі числа.");
            return;
        }

        if (n < 1 || n > MatrixSettings.MaxMatrixSize)
        {
            MessageBox.Show(
                $"Розмір матриці має бути від 1 до
{MatrixSettings.MaxMatrixSize}."
            );
            return;
        }

        if (min > max)
```

```
        {
            MessageBox.Show("Мінімальне значення не може перевищувати
максимальне.");
            return;
        }

        N = n;
        MinValue = min;
        MaxValue = max;

        DialogResult = true;
    }
}
```

MainWindow.xaml.cs (dir Object2)

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Text;
using System.Runtime.InteropServices;
using Lab6.Shared;

namespace Object2;
```

```
public partial class MainWindow : Window

{

    private readonly PipeCommandServer _server;



    public MainWindow()

    {

        InitializeComponent();



        _server = new PipeCommandServer(

            this,

            "Lab6_Object2",

            HandleSpecificCommandAsync

        );



        _ = _server.ListenAsync();

        Closed += (_, _) => _server.Stop();

    }



    private static int[,] CreateMatrix(int n, int min, int max)

    {

        int[,] matrix = new int[n, n];



        for (int i = 0; i < n; i++)

        {
```

```
            for (int j = 0; j < n; j++)
            {
                matrix[i, j] = (int)Random.Shared.NextInt64(min, (long)max
+ 1);
            }
        }

        return matrix;
    }

    private static string MatrixToText(int[,] matrix)
    {
        int n = matrix.GetLength(0);

        var text = new StringBuilder();
        text.AppendLine(n.ToString());

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (j > 0)
                {
                    text.Append(' ');
```

```
                }


                text.Append(matrix[i, j]);
            }
            // новий рядок
            text.AppendLine();
        }

        return text.ToString();
    }

    private void DisplayMatrix(int[,] matrix)
    {
        int n = matrix.GetLength(0);

        matrixGrid.Rows = n;
        matrixGrid.Columns = n;
        // видалити попередні
        matrixGrid.Children.Clear();

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
```

```
                var cell = new Border
                {
                    BorderBrush = Brushes.LightGray,
                    BorderThickness = new Thickness(1),
                    Background = Brushes.White,
                    MinWidth = 55,
                    MinHeight = 45,

                    Child = new TextBlock
                    {
                        Text = matrix[i, j].ToString(),
                        FontSize = 16,
                        HorizontalAlignment = HorizontalAlignment.Center,
                        VerticalAlignment = VerticalAlignment.Center
                    }
                };

                matrixGrid.Children.Add(cell);
            }
        }
    }

    public bool GenerateAndShow(int n, int min, int max)
    {
```

```
        if (n < 1 || n > MatrixSettings.MaxMatrixSize || min > max)
        {
            throw new ArgumentException("Некоректні параметри матриці.");
        }

        int[,] matrix = CreateMatrix(n, min, max);
        DisplayMatrix(matrix);

        try
        {
            Clipboard.SetText(MatrixToText(matrix));
        }
        catch (ExternalException)
        {
            infoText.Text = "Не вдалося записати матрицю до Clipboard.";
            return false;
        }

        infoText.Text =
            $"Матриця {n} × {n}, діапазон: [{min}; {max}]\n" +
            "Дані записано до Clipboard.";

        return true;
    }
```

```
private async Task<string> HandleSpecificCommandAsync(string command)
{
    string[] parts = command.Split('|');

    if (parts[0] != "GENERATE")
    {
        return "UNKNOWN_COMMAND";
    }

    if (parts.Length != 4 ||
        !int.TryParse(parts[1], out int n) ||
        !int.TryParse(parts[2], out int min) ||
        !int.TryParse(parts[3], out int max) ||
        n < 1 || n > MatrixSettings.MaxMatrixSize || min > max)
    {
        return "ERROR:PARAMETERS";
    }

    bool success = await Dispatcher.InvokeAsync(
        () => GenerateAndShow(n, min, max)
    );

    return success ? "READY" : "ERROR:CLIPBOARD";
```

```
    }

}
```

MainWindow.xaml.cs (dir Object3)

```
using System.Windows;

using System.Runtime.InteropServices;

using System.Numerics;

using Lab6.Shared;



namespace Object3;



public partial class MainWindow : Window

{

    private readonly PipeCommandServer _server;

    public MainWindow()

    {

        InitializeComponent();



        _server = new PipeCommandServer(

            this,

            "Lab6_Object3",

            HandleSpecificCommandAsync

        );
```

```
        _ = _server.ListenAsync();
        Closed += (_, _) => _server.Stop();
    }

    private static int[,] TextToMatrix(string text)
    {
        string[] lines = text.Split(
            new[] { '\r', '\n' },
            StringSplitOptions.RemoveEmptyEntries
        );

        if (lines.Length == 0 ||
            !int.TryParse(lines[0], out int n) ||
            n < 1 || n > MatrixSettings.MaxMatrixSize ||
            lines.Length != (long)n + 1)
        {
            throw new FormatException("Некоректний розмір матриці.");
        }

        int[,] matrix = new int[n, n];

        for (int i = 0; i < n; i++)
        {
            string[] numbers = lines[i + 1].Split(
```

```
                (char[]?)null,
                StringSplitOptions.RemoveEmptyEntries
            );

            if (numbers.Length != n)
            {
                throw new FormatException("Некоректна кількість
елементів.");
            }

            for (int j = 0; j < n; j++)
            {
                if (!int.TryParse(numbers[j], out matrix[i, j]))
                {
                    throw new FormatException("Матриця містить некоректні
числа.");
                }
            }
        }

        return matrix;
    }

    public bool ReadMatrixFromClipboard()
```

```
    {
        try
        {
            if (!Clipboard.ContainsText())
            {
                throw new FormatException("Clipboard не містить тексту.");
            }

            string text = Clipboard.GetText();
            int[,] matrix = TextToMatrix(text);
            BigInteger determinant = MatrixMath.Determinant(matrix);
            DisplayMatrix(matrix);

            resultText.Text = $"det(A) = {determinant}";

            return true;
        }
        catch (Exception ex) when (
            ex is FormatException or ExternalException)
        {
            matrixText.Clear();
            resultText.Text = $"Помилка: {ex.Message}";

            return false;
```

```
        }
    }

    private void DisplayMatrix(int[,] matrix)
    {
        int n = matrix.GetLength(0);

        var text = new System.Text.StringBuilder();

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                text.Append(matrix[i, j].ToString().PadLeft(6));
            }

            text.AppendLine();
        }

        matrixText.Text = text.ToString();
    }

    private async Task<string> HandleSpecificCommandAsync(string command)
    {
```

```
        if (command != "CALCULATE")
        {
            return "UNKNOWN_COMMAND";
        }

        bool success = await Dispatcher.InvokeAsync(
            () => ReadMatrixFromClipboard()
        );

        return success ? "READY" : "ERROR:DATA";
    }
}
```

MatrixMath.cs (dir Object3)

```
using System.Numerics;

namespace Object3;

internal static class MatrixMath
{

    // алгоритм Барейса (без використання чисел із плаваючою комою)
    internal static BigInteger Determinant(int[,] matrix)
    {
```

```
        int n = matrix.GetLength(0);


        BigInteger[,] a = new BigInteger[n, n];


        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                a[i, j] = matrix[i, j];
            }
        }


        BigInteger previousPivot = 1;
        int sign = 1;


        for (int k = 0; k < n - 1; k++)
        {
            int pivotRow = k;


            while (pivotRow < n && a[pivotRow, k] == 0)
            {
                pivotRow++;
            }
```

```
            if (pivotRow == n)
            {
                return BigInteger.Zero;
            }

            if (pivotRow != k)
            {
                for (int j = 0; j < n; j++)
                {
                    (a[k, j], a[pivotRow, j]) =
                        (a[pivotRow, j], a[k, j]);
                }

                sign = -sign;
            }

            BigInteger pivot = a[k, k];

            for (int i = k + 1; i < n; i++)
            {
                for (int j = k + 1; j < n; j++)
                {
                    a[i, j] =
                        (a[i, j] * pivot - a[i, k] * a[k, j])
```

```
                        / previousPivot;
                }
            }

            for (int i = k + 1; i < n; i++)
            {
                a[i, k] = 0;
            }

            previousPivot = pivot;
        }

        return sign * a[n - 1, n - 1];
    }
}
```

MatrixSettings.cs (dir Shared)

```
namespace Lab6.Shared;


internal static class MatrixSettings
{
    internal const int MaxMatrixSize = 100;
}
```

PipeCommandServer.cs (dir Shared)

```
using System.Diagnostics;

using System.IO;

using System.IO.Pipes;

using System.Windows;



namespace Lab6.Shared;



internal sealed class PipeCommandServer

{

    private readonly Window _window;

    private readonly string _pipeName;

    private readonly Func<string, Task<string>> _handleCommand;



    private readonly CancellationTokenSource _stop = new();



    internal PipeCommandServer(

    Window window,

    string pipeName,

    Func<string, Task<string>> handleCommand)

    {

        _window = window;

        _pipeName = pipeName;

        _handleCommand = handleCommand;
```

```
    }


    private async Task<string> ProcessCommandAsync(string? command)

    {

        if (command == "PING")

        {

            return "PONG";

        }


        if (command == null)

        {

            return "ERROR:EMPTY_COMMAND";

        }


        if (command.StartsWith("MOVE|"))

        {

            string[] values = command.Split('|');


            if (values.Length != 5 ||

                !int.TryParse(values[1], out int x) ||

                !int.TryParse(values[2], out int y) ||

                !int.TryParse(values[3], out int width) ||

                !int.TryParse(values[4], out int height) ||

                width <= 0 || height <= 0)
```

```
            {
                return "ERROR:POSITION";
            }

            await _window.Dispatcher.InvokeAsync(() =>
            {
                _window.WindowState = WindowState.Normal;
                _window.Left = x;
                _window.Top = y;
                _window.Width = width;
                _window.Height = height;
            });

            return "OK";
        }

        return await _handleCommand(command);
    }

    internal async Task ListenAsync()
    {
        while (!_stop.IsCancellationRequested)
        {
            try
```

```
        {
            using var pipe = new NamedPipeServerStream(
                _pipeName,
                PipeDirection.InOut,
                1,
                PipeTransmissionMode.Byte,
                PipeOptions.Asynchronous
            );

            await pipe.WaitForConnectionAsync(_stop.Token);

            using var reader = new StreamReader(pipe);

            using var writer = new StreamWriter(pipe)
            {
                AutoFlush = true
            };

            string? command = await reader.ReadLineAsync(
                _stop.Token
            );

            if (command == "QUIT")
            {
```

```
                await writer.WriteLineAsync("BYE");

                _ = _window.Dispatcher.BeginInvoke(
                    new Action(_window.Close)
                );

                return;
            }

            string response = await ProcessCommandAsync(command);

            await writer.WriteLineAsync(response);
        }
        catch (OperationCanceledException)
        {
            return;
        }
        catch (IOException ex)
        {
            Debug.WriteLine(
                $"Помилка каналу {_pipeName}: {ex.Message}"
            );

            await Task.Delay(200);
```

```
            }

        }

    }



    internal void Stop()

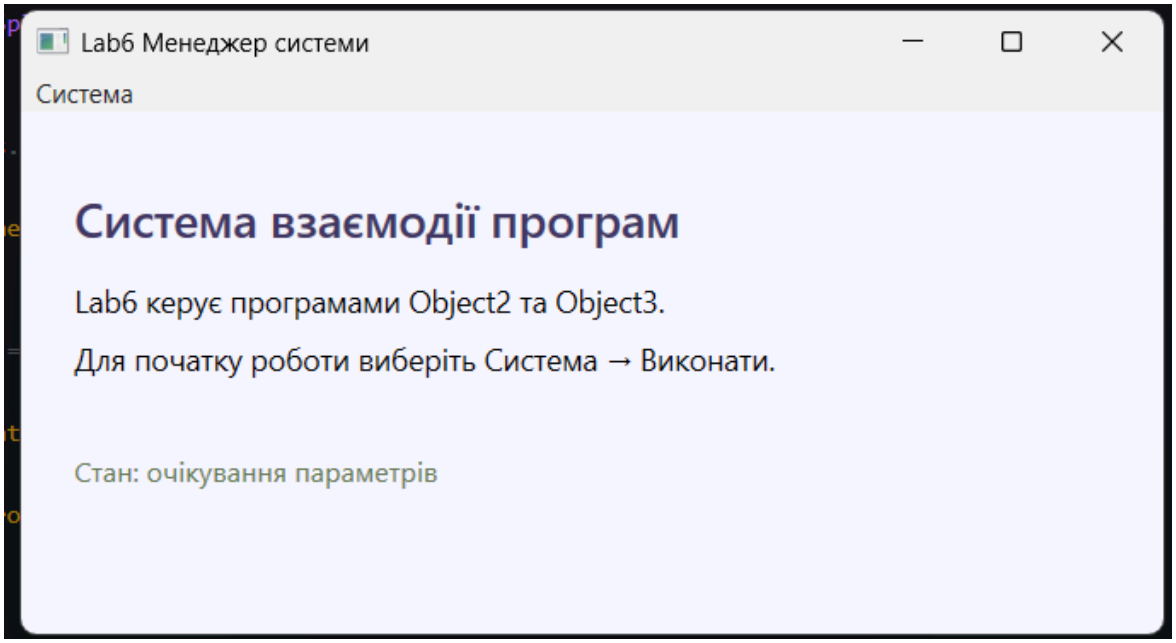
    {

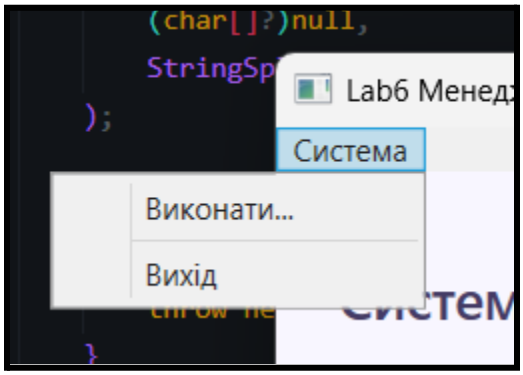
        _stop.Cancel();

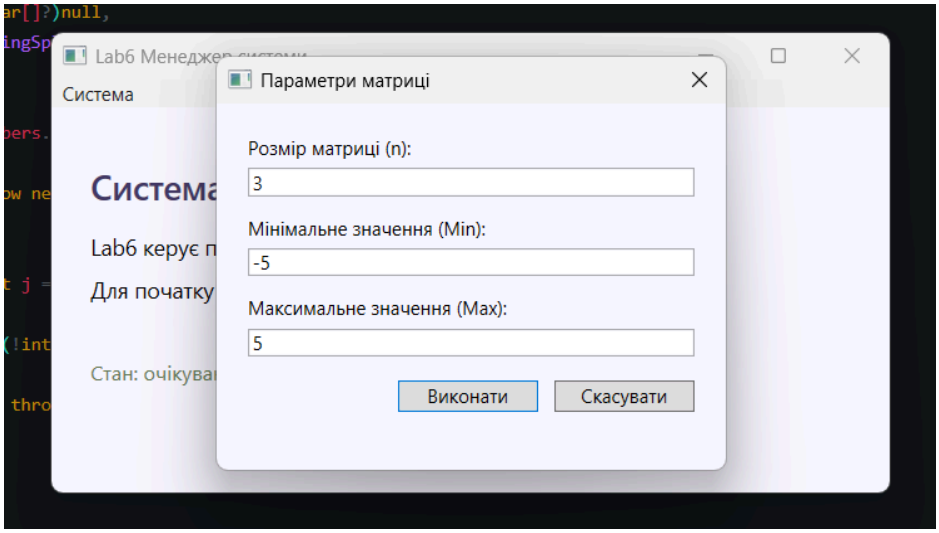
    }

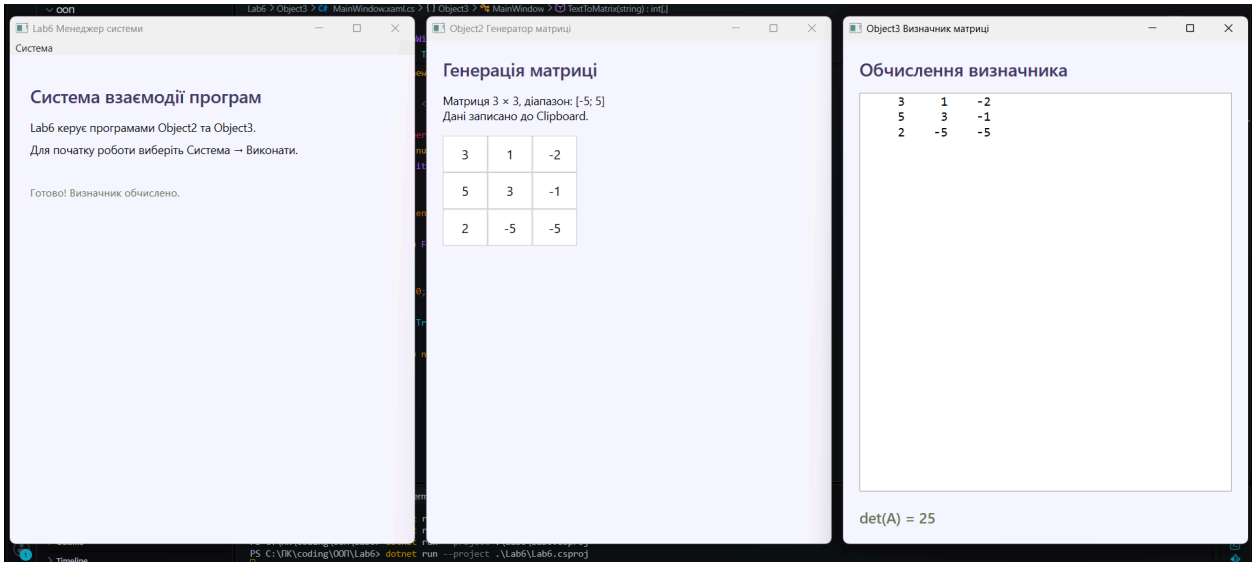
}
```

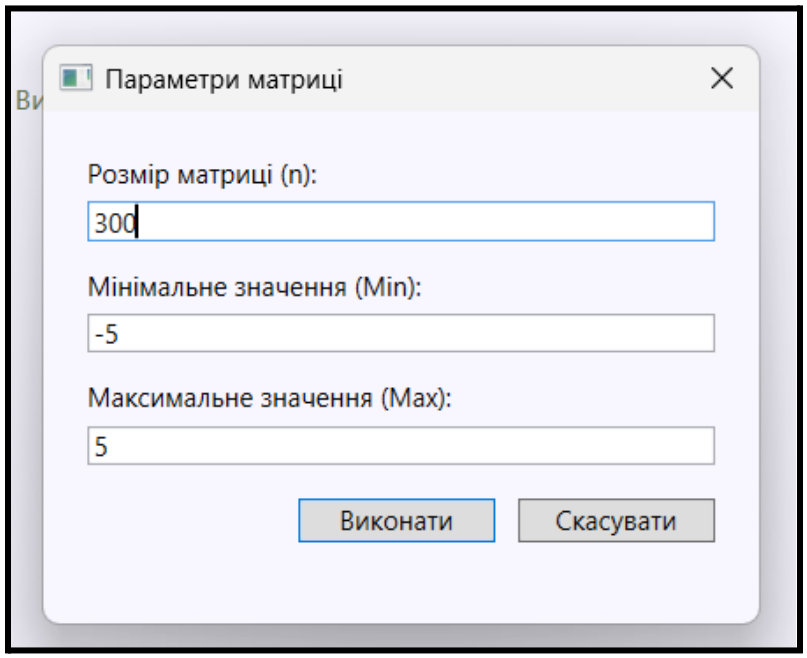
Скріншоти

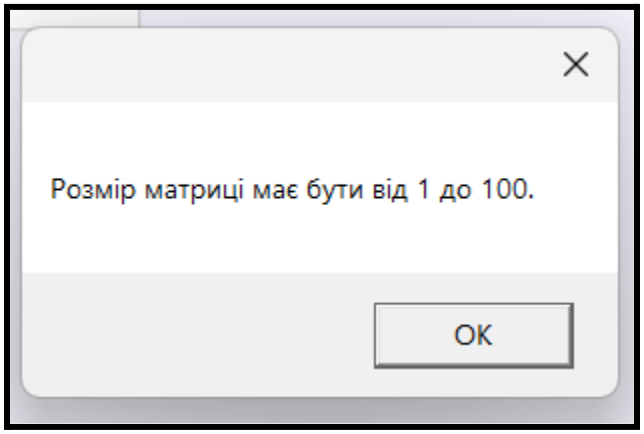

Lab6 Менеджер системи
Система
Система взаємодії програм
Lab6 керує програмами Object2 та Object3.
Для початку роботи виберіть Система → Виконати.
Стан: очікування параметрів

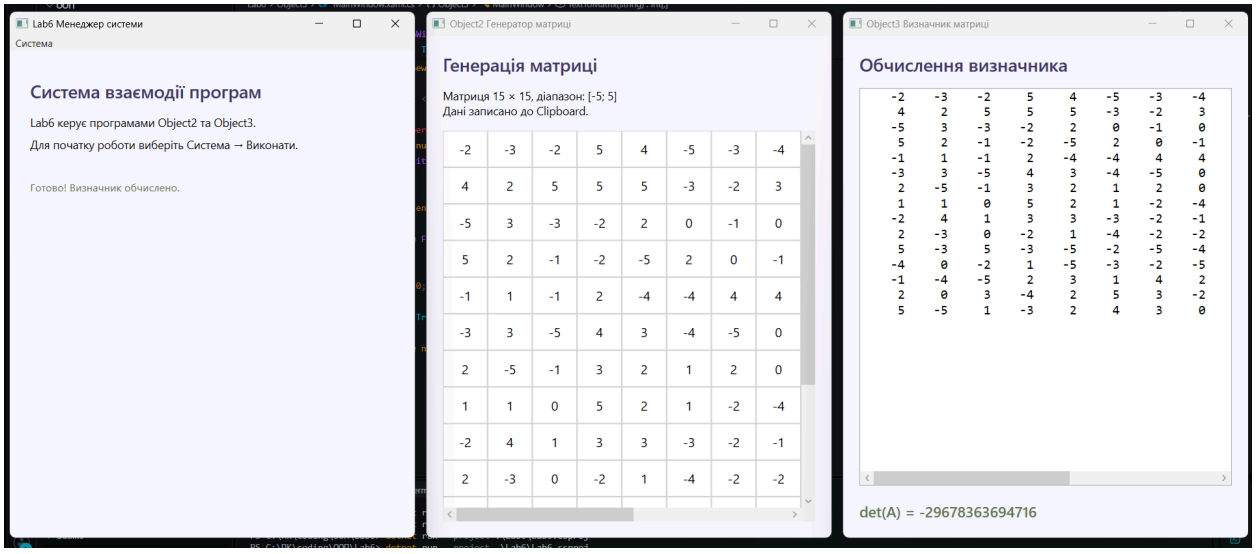

Lab6 Менедж
Система
Виконати...
Вихід


Система
Параметри матриці
Розмір матриці (n):
3
Мінімальне значення (Min):
-5
Максимальне значення (Max):
5
Виконати
Скасувати


Lab6 Менеджер системи
Система
Система взаємодії програм
Lab6 керує програмами Object2 та Object3.
Для початку роботи виберіть Система → Виконати.
Готово! Визначник обчислено.
Object2 Генератор матриці
Генерація матриці
Матриця 3 × 3, діапазон: [-5; 5]
Дані записано до Clipboard.
3 1 -2
5 3 -1
2 -5 -5
Object3 Визначник матриці
Обчислення визначника
3 1 -2
5 3 -1
2 -5 -5
det(A) = 25


Параметри матриці
Розмір матриці (n):
300
Мінімальне значення (Min):
-5
Максимальне значення (Max):
5
Виконати
Скасувати


Розмір матриці має бути від 1 до 100.
OK


Lab6 Менеджер системи
Система
Система взаємодії програм
Lab6 керує програмами Object2 та Object3.
Для початку роботи виберіть Система → Виконати.
Готово! Визначник обчислено.
Object2 Генератор матриці
Генерація матриці
Матриця 15 × 15, діапазон: [-5; 5]
Дані записано до Clipboard.
-2 -3 -2 5 4 -5 -3 -4
4 2 5 5 5 -3 -2 3
-5 3 -3 -2 2 0 -1 0
5 2 -1 -2 -5 2 0 -1
-1 1 -1 2 -4 -4 4 4
-3 3 -5 4 3 -4 -5 0
2 -5 -1 3 2 1 2 0
1 1 0 5 2 1 -2 -4
-2 4 1 3 3 -3 -2 -1
2 -3 0 -2 1 -4 -2 -2
Object3 Визначник матриці
Обчислення визначника
-2 -3 -2 5 4 -5 -3 -4
4 2 5 5 5 -3 -2 3
-5 3 -3 -2 2 0 -1 0
5 2 -1 -2 -5 2 0 -1
-1 1 -1 2 -4 -4 4 4
-3 3 -5 4 3 -4 -5 0
2 -5 -1 3 2 1 2 0
1 1 0 5 2 1 -2 -4
-2 4 1 3 3 -3 -2 -1
2 -3 0 -2 1 -4 -2 -2
5 -3 5 -3 -5 -2 -5 -4
-4 0 -2 1 -5 -3 -2 -5
-1 -4 -5 2 3 1 4 2
2 0 3 -4 2 5 3 -2
5 -5 1 -3 2 4 3 0
det(A) = -29678363694716

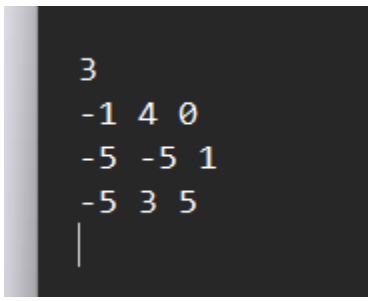
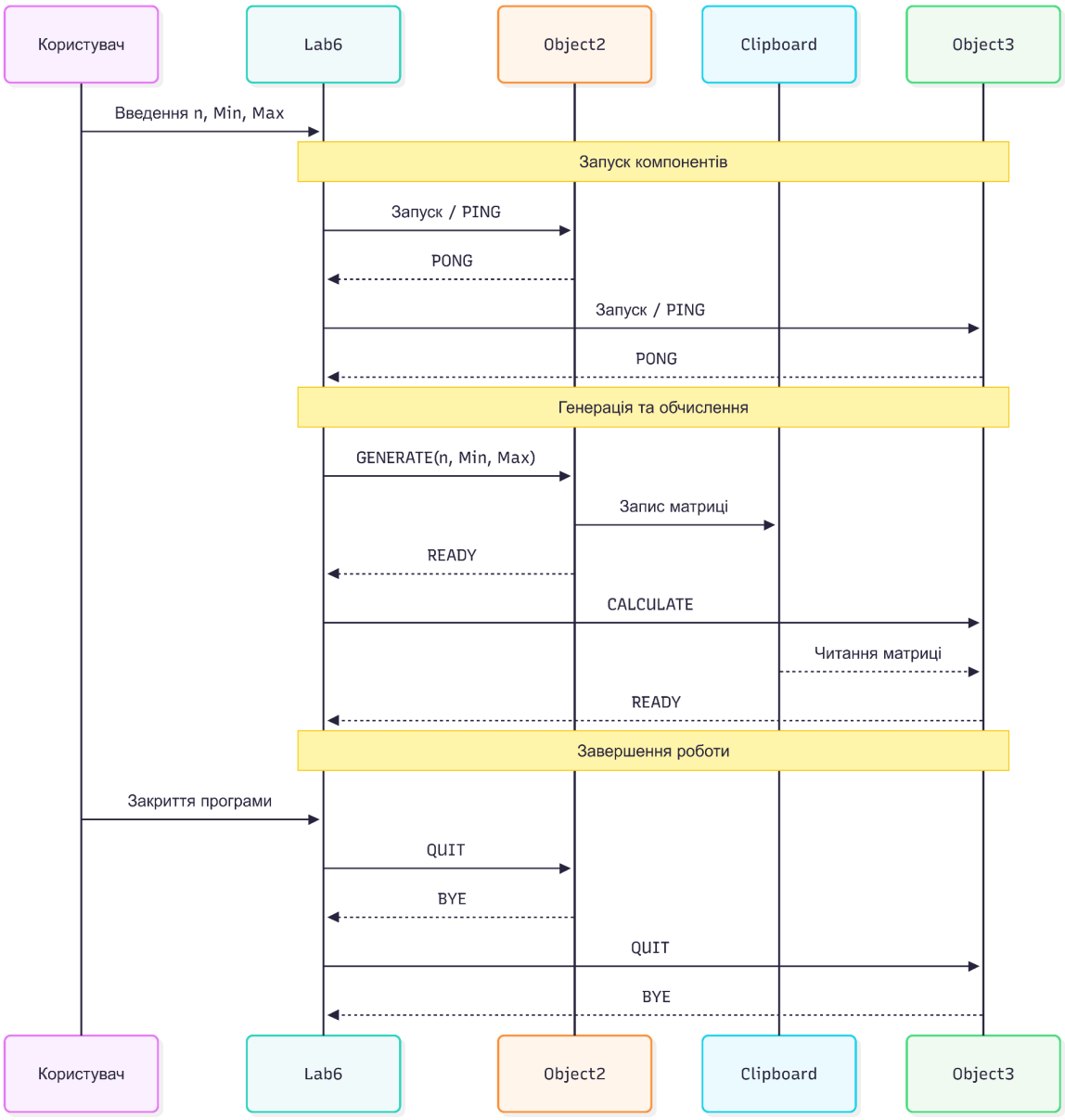

3
-1 4 0
-5 -5 1
-5 3 5

Схема послідовності надсилання-обробки повідомлень:



Висновки:

У ході лабораторної роботи було розроблено систему з трьох незалежних програм, які взаємодіють між собою за допомогою Named Pipes та буфера обміну Windows. Реалізовано автоматичний запуск і завершення процесів, генерацію матриці та обчислення її визначника. Також забезпечено обробку команд і синхронізацію роботи компонентів. У результаті було закріплено практичні навички організації міжпроцесної взаємодії в C# та WPF.